Daily Scholar Papers Report — 2026-08-15

2026-08-15

Daily Scholar Papers Report — 2026-08-15

Download PDF

Window covered: 2026-08-14 → 2026-08-15 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

After eleven days in which the intake channel produced almost nothing, today's window delivered fifteen candidates across four alert threads — the largest single-day intake this digest has seen since the spring. Nine cleared Stage-1 screening. The user-curated self-email queue was silent, so everything below arrived through Scholar.

Two results stand out, and they stand out for the same underlying reason: both take a mature, well-understood analysis engine and put an LLM *outside* it rather than inside it. **Agolic** leaves symbolic execution's state selection entirely alone and instead has an agent decide how the tool should be invoked from one bounded run to the next — and gets more than 3× the branch coverage of continuous symbolic execution on every program tested. **PDFuzzer** does not ask a model to mutate inputs; it asks a model to read the JavaScript API manuals and build grammars and inter-API relations, then hands those to a constraint solver — and finds 31 zero-days across Adobe Acrobat Reader, Foxit, and PDF-XChange. Neither paper's LLM is doing the search. In both cases it is doing the *specification work* that used to be done by a graduate student with a manual, and the classical engine underneath is unmodified. That is a sharper and more transferable design pattern than the current default of pointing a model at the artefact and asking it for an answer.

Beneath those, the Keep tier is unusually coherent: false-positive reduction for SAST via an evolving memory base, model-level (rather than inference-time) hardening for secure code generation, a benchmark that instruments *where* coding agents fail rather than just whether they fail, a reference-free LLM-as-a-Judge scheme for binary reverse engineering, and a fault-injection harness for agent systems. Four of the five come from one followed research group, which is worth naming as a sampling fact rather than a quality signal.

One methodological note carries across the day. Several papers in this window report figures at or near ceiling on their chosen benchmark. Those numbers are reported below as the papers state them, but a near-perfect score on a curated benchmark is a statement about the benchmark as much as about the method, and the honest reading of the SWE-RPG result — 31.5% average resolved rate for production coding agents — is that when the evaluation is designed to be diagnostic rather than summary, the numbers get a great deal less flattering.

Outstanding: 2 · **Keep:** 5 · **Borderline High-Priority:** 2

Highlighted Papers

Title	Authors	Venue	Link
Agentic Planning for Symbolic Execution	D. K. J. Yang, Y. Noller, C. S. Păsăreanu, Y. Sun	arXiv preprint (cs.PL), 2026	arXiv:2608.06397
From Documentation to Zero-day Vulnerabilities: LLM-Driven Fuzzing of JavaScript Engines in PDF Readers	S. Guo, S. Pletinckx, T. Yu, Y. Kaya, S. Ullah, W. Guo, C. Kruegel, G. Vigna	ACM CCS 2026	arXiv:2608.06641
Memoir: Learning, Verifying, and Evolving False-Positive Memories for SAST Tools	S. Guan, Q. Hou, Y. Chen, X. Wen, J. Feng, K. Lian, C. Gao	arXiv preprint, 2026	arXiv:2608.09181
Understanding and Improving Model Editing for Secure Code Generation	W. Sun, Q. Zhang, Y. Chen, C. Yang, G. Tan, D. Lo	ISSTA 2026	arXiv:2608.06848
A Unified Issue Resolution Benchmark for Requirement Clarification, Planning, and Code Generation for Coding Agents	X. Zhou, C. Y. Chong, K. Kim, Y. Peng, R. Shu, Z. Wu, X. Han, G. Yuan, Z. Zhuang, J. Kim, J. Ju, S. Ju, T. Yoon, D. Lo	arXiv preprint, 2026	arXiv:2608.09072
Beyond Text Matching: Towards Reference-Free Evaluation for Human-Oriented Binary Reverse Engineering	X. Shang, L. Hu, X. Jiang, J. Shi, J. He, Z. Yang, S. Cheng, G. Chen, W. Zhang, D. Lo	ASE 2026	arXiv:2608.07038
AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection	G. Tan, Z. Sun, J. Shi, T. Zhang, Z. He, Q. Wu, S. Liang, et al.	arXiv preprint, 2026	arXiv:2608.06790
SmellCC: A Tool for Automated Code Smells Remediation	X. Zhang, Y. Zhang, Z. Gao, X. Hu, X. Xia	arXiv preprint, 2026	arXiv:2608.09477
How Reasoning Shapes Social Bias in LLM-Generated Code?	W. Sun, J. Shi, Z. Yang, Y. Chen, H. Li, M. Yan, D. Lo	arXiv preprint, 2026	arXiv:2608.06829

Papers

1.1 · SYMBOLIC EXECUTION · Agolic plans how the tool is used across bounded runs and covers 3× more branches than continuous symbolic execution👍😐💉

Agentic Planning for Symbolic Execution Authors: Daniel Koh Ji Yang, Yannic Noller, Corina S. Păsăreanu, Youcheng Sun **Venue:** arXiv preprint arXiv:2608.06397 (cs.PL; cs.AI, cs.SE), submitted 31 Jul 2026 https://arxiv.org/abs/2608.06397 Licence: CC BY-NC-SA 4.0.

The idea, and why the framing is the contribution

Almost every attempt to improve symbolic execution over the last two decades has reached inside the engine: better search heuristics, better state merging, better constraint caching, better path prioritisation. The LLM-flavoured versions of that programme do the same thing with a model in the loop — let the model pick the next state, let the model prune, let the model summarise a path condition.

This paper does the opposite, and says so plainly. It leaves "ordinary state exploration to the underlying tool" and moves the intelligence up one level, to the question of *how the same tool is invoked from one bounded run to the next*. The system, **Agolic**, is described as "an agentic planning system that uses evidence from earlier runs to choose and configure later bounded symbolic execution (BSE) runs, which the underlying symbolic execution tool then carries out."

The distinction matters more than it might sound. A symbolic execution engine is a highly tuned artefact whose correctness properties depend on its internal invariants; putting a language model inside the state-selection loop puts those invariants at the mercy of a component with no soundness story. Putting the model *outside*, in the position of deciding budgets, targets, and configuration for the next bounded run, leaves the engine's guarantees untouched and confines the model's role to something it is demonstrably good at — reading source code and reasoning about which unreached region is worth aiming at next, given what the last few attempts did.

The loop

The evaluated adaptation targets branch-coverage exploration. The agent reasons over three kinds of evidence: the program's source code, replayed coverage from earlier runs, and the record of earlier targeting attempts. That third input is the interesting one. It means the planner has access not only to *what is still uncovered* but to *what has already been tried and failed*, which is precisely the information a continuous run throws away when it exhausts its budget.

The paper is explicit that the planner is a parameter rather than a fixture: "The planning intelligence, available evidence and execution modes can be adapted to the symbolic execution tool and analysis objective." Branch coverage is one instantiation; the architecture is stated as general over analysis objectives.

Headline numbers

Evaluated on several C and C++ programs (seven, per the comparison-corpus result):

- On **every** program, Agolic extends the branch coverage obtained by continuous symbolic execution, covering **more than 3× as many branches on average**.
- It covers more branches than **each individual** comparison corpus from coverage-guided fuzzing and from compiler-based concolic execution.
- It reaches branches **absent from all comparison corpora combined** on **six of the seven** programs.

That last figure is the one to hold onto. Beating a baseline on aggregate coverage is a routine result; reaching regions that the union of fuzzing and concolic execution never touched is a claim about *complementarity* rather than dominance, and complementarity is what makes a technique worth adding to a pipeline rather than swapping into one.

What to be careful about

Seven C/C++ programs is a small evaluation, and the paper does not claim otherwise. Two questions a full reading should press on: how much of the 3× is attributable to the agent's reasoning versus simply to the restart-and-retarget structure — a well-tuned non-LLM scheduler over bounded runs is the obvious ablation — and what the wall-clock and token cost of the planning loop is relative to just giving the continuous run a proportionally larger budget. Neither is answered in the abstract, and the digest's assessment here is based on the abstract, metadata, and licence page rather than a full-text read; the numbers quoted above are the paper's own.

The authors' own closing framing is that the results "point to considerable untapped potential in existing symbolic execution tools."

Why it is the pick of the day

Because the reusability is unusually clean. Nothing in the architecture is specific to branch coverage, to C, or to any particular engine. Any analysis that (a) runs under a resource bound, (b) leaves a legible record of what it reached, and (c) accepts configuration for a targeted re-run fits the same template — bounded model checking, targeted fuzzing campaigns, taint analysis with a call-depth cap. The paper hands over a pattern, not just a tool.

1.2 · FUZZING · PDFuzzer reads the API manual to build grammars, then solves for call sequences — 48% more coverage and 31 zero-days in Acrobat, Foxit, and PDF-XChange👍😐🩹

From Documentation to Zero-day Vulnerabilities: LLM-Driven Fuzzing of JavaScript Engines in PDF Readers Authors: Suyue Guo, Stijn Pletinckx, Tianle Yu, Yigitcan Kaya, Saad Ullah, Wenbo Guo, Christopher Kruegel, Giovanni Vigna **Venue:** ACM CCS 2026 (16 pages, 2 figures) https://arxiv.org/abs/2608.06641 Licence: CC BY 4.0.

The gap being closed

PDF readers embed full JavaScript engines with large, vendor-specific API surfaces. Existing fuzzers for these engines, the paper argues, "rely on simple test cases that involve only individual API calls, leading to limited coverage and potentially missing vulnerabilities that require sequences of API calls."

This is a real and specific structural problem, not a generic complaint about coverage. A bug that requires `A()` to place the document in a particular state before `B()` is reachable is invisible to any generator that emits one call at a time, no matter how many times it is run. The bug class is defined by the *sequence*, and single-call fuzzing has zero probability of finding it.

The pipeline

PDFuzzer is a three-stage design, and the division of labour is the point:

1. **LLM as specification extractor.** A large language model constructs context-free grammars and infers relationships between individual API calls, from two sources: the JavaScript API

manuals and execution traces. Documentation is doing real work here — it is the artefact that describes what the API *means*, and it has historically been read by humans and then encoded by hand into a fuzzer's grammar.

2. **Constraint solver as generator.** Given the grammars and the inferred relations, "PDFuzzer employs a constraint solver to generate concrete API call sequences for fuzzing." The model does not emit test cases. It emits a *specification*; a deterministic solver emits the test cases.
3. **Fuzzing loop** against the target readers.

The reason this split is worth copying is that it puts the LLM where its failure mode is tolerable and the solver where determinism is needed. A hallucinated grammar production is a wasted branch of the search space; a hallucinated test case is a wasted execution *and* a corrupted coverage signal. Confining generation to the solver means the unsound component's errors are bounded and visible.

Headline numbers

Against state-of-the-art PDF fuzzers (TypeOracle, Favocado, Cooper) and LLM-based fuzzers (Fuzz4All, naive LLM), on Adobe Acrobat Reader, Foxit PDF Reader, and PDF-XChange Editor:

- **Up to 48% higher coverage** than existing tools.
- **31 zero-day vulnerabilities** identified across the three readers, ranging "from information leakage to arbitrary code execution."
- Ablation confirms each component is necessary; the LLM stages achieve **93–98% accuracy** across all pipeline stages.
- All vulnerabilities were disclosed to vendors via coordinated disclosure, and the authors received bug bounties.

The 93–98% figure deserves a moment. It is the number that makes the architecture defensible: it says the specification-extraction step is reliable enough that the solver downstream is mostly working from correct grammars, and the remaining 2–7% degrades gracefully into wasted search rather than into false results.

Assessment

Thirty-one zero-days in three widely deployed commercial readers is not a benchmark result — it is a field result, with coordinated disclosure and bounties as external validation that the findings were real and exploitable. That is a substantially stronger form of evidence than a coverage delta on a synthetic suite, and it is the reason this paper sits in the Outstanding tier alongside a methodologically cleaner but less field-tested result.

The transferable lesson is the documentation-as-input move. Any target with a large documented API and a stateful object model — browser DOM, database client libraries, cloud SDKs, kernel syscall interfaces with man pages — presents the same opportunity, and the same reason to keep the model on the specification side of the boundary.

This digest's assessment is based on the abstract, venue metadata, and licence page; quoted figures are the paper's own.

2.1 · STATIC ANALYSIS · Memoir turns historical false-positive alerts into an evolving semantic memory base, reporting F1 99.43% on CWE-Bench-Java

Memoir: Learning, Verifying, and Evolving False-Positive Memories for Static Application Security Testing Tools Authors: Shenyuan Guan, Qiaodan Hou, Yanjun Chen, Xincheng Wen, Jia Feng, Keke Lian, Cuiyun Gao **Venue:** arXiv preprint arXiv:2608.09181 (cs.SE; cs.CR), submitted 10 Aug 2026 https://arxiv.org/abs/2608.09181 Licence: arXiv non-exclusive distribution licence — no figures embedded.

The problem, stated precisely

SAST false positives are an old complaint, but the paper's diagnosis of why existing FP-reduction methods underperform is specific and correct. Two challenges: first, "the large differences among SAST tools and vulnerability categories make it difficult for these methods to learn recurring patterns in historical false positives"; second, "the knowledge used by these methods are largely static and cannot be updated as newly validated cases accumulate."

The second is the more interesting one, and it is the one most FP-reduction work quietly ignores. A classifier trained once on a snapshot of triaged alerts is stale the moment a team's codebase or coding conventions shift, and the whole value proposition of FP suppression — restoring developer trust — collapses if the suppressor's accuracy silently decays.

Architecture

Two modules:

Historical semantic memory construction. Historical FP alerts are converted "into structured semantic memories through LLM-guided annotation, pattern clustering, and memory synthesis to capture reusable behavioral patterns." Note the three-step compression: annotate individual alerts, cluster them into patterns, synthesise a memory. The clustering step is what makes the memory base sublinear in alert count and, in principle, transferable across tools.

Memory-driven identification and evolution. At prediction time the system retrieves relevant memories and "performs semantic verification against taxonomy consistency and security invariants before making the final prediction." Verified predictions are then written back into the memory repository.

That verification gate is the design decision worth stealing. Retrieval-augmented classification without a check is a machine for amplifying whatever bias is already in the retrieved examples; gating write-back on consistency with an explicit taxonomy and with security invariants gives the feedback loop a fixed point that is not simply "whatever the model said last time."

Headline numbers

On CWE-Bench-Java: **F1-score 99.43%**, **Recall 98.88%**, and **perfect Precision**, reported as consistently outperforming baselines. An industrial case study on production systems at a large IT company reports that the learned memory base "generalizes effectively across different SAST tools without retraining."

Reading the numbers honestly

Perfect precision and 99.43% F1 on a curated benchmark is a result that should raise a question rather than settle one. CWE-Bench-Java is a fixed, well-studied dataset; a memory-based method that has seen historical alerts from the same distribution is in an unusually favourable position, and the benchmark's own label noise puts a ceiling on how meaningful the last fraction of a percent can be.

None of this is a criticism of the work — it is the standard difficulty of evaluating FP suppression, where the ground truth is itself a human triage judgement.

The industrial cross-tool generalisation study is, for that reason, the more load-bearing evidence in the paper, and the part a full read should scrutinise first: how many tools, how many alerts, and what the precision looked like when the memory base met a SAST tool it had never been trained against.

2.2 · SECURE CODE GEN · Model editing beats inference-time hardening on seen CWEs by 15–25% but transfers poorly; SafeEdit recovers 11.7–15.5 pp of Pass@1

Understanding and Improving Model Editing for Secure Code Generation Authors: Weifeng Sun, Quanjun Zhang, Yuchen Chen, Chengran Yang, Gou Tan, David Lo **Venue:** ISSTA 2026 https://arxiv.org/abs/2608.06848 Licence: CC BY 4.0.

The question

Hardening an LLM against generating vulnerable code has mostly been done at inference time — decoding-time steering, auxiliary security models, prompt scaffolds. That works without touching the model, but "relies on auxiliary components and adds runtime overhead." This paper asks whether the alternative — *editing the model's weights* — is a better trade, and it is billed as the first systematic study of model editing as a model-level hardening mechanism for secure code generation.

The framing is well-posed because it isolates a real engineering decision. Inference-time hardening is a permanent tax on every request; model editing is a one-time cost. If editing works, it is strictly preferable at deployment scale. The paper's job is to find out whether it works, and the answer is a qualified no that becomes a qualified yes.

Findings

Three editing methods, evaluated across diverse LLM families against CoSec (a representative inference-time approach), on four axes: security, robustness, generalisation, functional correctness.

- **On seen vulnerability types**, model editing beats CoSec, improving security ratios by **15–25%** over vanilla models, "with gains remaining stable under prompt perturbations." Robustness to perturbation is a meaningful check — a hardening method that evaporates when the prompt is reworded is not hardening.
- **But** the improvements "transfer unreliably to unseen vulnerabilities and can reduce functional correctness."

That second bullet is the honest core of the paper and the reason it is worth reading rather than skimming. Editing a model to stop emitting a specific vulnerable pattern is, mechanistically, closer to memorising a patch than to installing a security concept — and the generalisation failure is exactly what that mechanistic story predicts. The correctness regression is the familiar edit-locality problem showing up in a domain where it has real consequences.

SafeEdit

The proposed fix is **SafeEdit**, a post-edit refinement method combining functional tuning with edit-aware regularization. Across eight target LLMs:

- Pass@1 improves over UltraEdit by **11.73 / 13.70 / 15.50 percentage points** at temperature **T = 0.1 / 0.4 / 0.8**, "while largely preserving security."
- Relative security-ratio gains of **7.54%–12.04%** compared with CoSec.
- Evaluation on CodeGuard+ confirms improved joint secure-and-correct generation.
- SafeEdit and CoSec are complementary; combining them further improves security while maintaining functional correctness.

The temperature sweep is a good sign. Reporting Pass@1 at a single temperature is how correctness regressions get hidden, and the gains hold — indeed grow — as sampling gets noisier.

Why it matters beyond secure code

The complementarity result is the practical takeaway: model-level and inference-time hardening are not competing answers to one question but two layers that compose. For anyone shipping a code model, that reframes the decision from "which" to "how much of each," and the paper offers evidence-backed guidance for that choice rather than a single recommended configuration.

2.3 · CODING AGENTS · SWE-RPG instruments where agents fail: 31.5% average resolved rate, with implicit-requirement recovery the bottleneck in 24.5–46.0% of runs👍😐💉

A Unified Issue Resolution Benchmark for Requirement Clarification, Planning, and Code Generation for Coding Agents Authors: Xin Zhou, Chun Yong Chong, Kisub Kim, Yun Peng, Rui Shu, Zihan Wu, Xu Han, Guowen Yuan, Zeyang Zhuang, Jounghoon Kim, Jeongjin Ju, Seongmin Ju, Taein Yoon, David Lo **Venue:** arXiv preprint arXiv:2608.09072 (cs.SE; cs.AI), submitted 10 Aug 2026 https://arxiv.org/abs/2608.09072 Licence: CC BY 4.0. Artefacts: https://github.com/Xin-Zhou-smu/SWE-RPG-Bench

The critique of pass/fail

The paper's opening argument is the sharpest statement of a problem the SWE-bench family has had since the beginning: "A pass/fail outcome cannot characterize how an unsuccessful trajectory diverges from the requirements and implementation process needed for a correct patch."

Satisfying a repository-level request is a chain — recover explicit and implicit requirements, formulate a repository-grounded plan, translate it into code. A binary outcome collapses that chain into one bit, which means a benchmark score can go up without anyone learning which link was broken. For a field trying to *improve* agents rather than rank them, that is close to useless.

The construction

SWE-RPG pairs executable patch evaluation with validated ground-truth references for two intermediate stages: (1) Requirement Clarification and (2) Implementation Planning. These intermediate GTs "support retrospective, GT-aligned diagnosis of complete coding-agent trajectories across clarification, planning, code generation, and artifact submission."

Scale: **163 tasks** drawn from **31 Python and Java repositories** — **113 bug fixes** and **50 feature additions**. Including feature additions matters; the bug-fix monoculture of earlier benchmarks systematically under-tests exactly the requirement-recovery skill this benchmark is built to measure.

Headline numbers

Three coding agents (Claude Code, Codex, OpenCode) across six LLM backends (including Claude-Sonnet-5 and GPT-5.6-Terra):

- Average resolved rate: **31.5%**.
- Intermediate-GT diagnosis identifies **implicit requirement recovery** as the main bottleneck, accounting for **24.5%–46.0%** of agent runs.

Why this is the most actionable result in the window

The 31.5% is a useful corrective, but the diagnostic finding is the contribution. If between a quarter and nearly half of failures trace to the agent not recovering requirements that were never stated explicitly, then a large fraction of current effort — better retrieval, better patch generation, longer context — is aimed at the wrong link in the chain. The failure is upstream of code generation entirely: the agent produces a competent patch for a problem nobody asked it to solve.

That also suggests a cheap intervention the benchmark makes measurable for the first time: agents that ask a clarifying question, or that explicitly enumerate inferred requirements before planning, should move this number in a way that is now attributable rather than merely correlated with a score bump.

The caveat is the usual one for intermediate ground truth — the clarification and planning references are themselves human judgements, and "GT-aligned" diagnosis inherits whatever variance those judgements carry. The paper describes the references as validated; a full read should check how.

2.4 · BINARY ANALYSIS · BinJudge routes per-sample judge configurations: 63.20% human correlation vs 35.04% for classical metrics, at 0.06–0.84× the API cost👍😐🩹

Beyond Text Matching: Towards Reference-Free Evaluation for Human-Oriented Binary Reverse Engineering Authors: Xiuwei Shang, Li Hu, Xiao Jiang, Jieke Shi, Junda He, Zhou Yang, Shaoyin Cheng, Guoqiang Chen, Weiming Zhang, David Lo **Venue:** ASE 2026 (41st IEEE/ACM International Conference on Automated Software Engineering) https://arxiv.org/abs/2608.07038 Licence: CC BY 4.0.

The measurement crisis this addresses

Human-Oriented Binary Reverse Engineering turns decompiled pseudocode into something a human can read. The field has a measurement problem that is worse than the usual one, and the paper enumerates it precisely: human evaluation does not scale; execution-based metrics "require executable test cases and runtime environments that are often unavailable for real-world binaries"; and reference-based metrics "rely on high-quality source code references that are typically inaccessible and fail to capture semantically equivalent but lexically diverse outputs."

That third clause is the crux. The whole point of HOBRE output is that it should read differently from — and better than — whatever reference exists. Scoring it by textual overlap with a reference penalises the behaviour the technique is trying to produce. Text-matching metrics in this domain are not merely noisy; they are pointed the wrong way.

The study and the artefact

Across three representative tasks — function name recovery, binary code summarization, decompilation optimization — the paper presents the first systematic investigation of LLM-as-a-Judge for HOBRE, and introduces **BinJudgeBench**, described as the first expert-annotated, reference-free evaluation benchmark based on multi-dimensional human judgment.

- LLM-as-a-Judge achieves an average correlation of **63.20%** with human judgment, against **35.04%** for traditional automated metrics.
- Sweeping backbone LLMs, prompting strategies, and decoding temperatures, the paper finds that no "one-size-fits-all" configuration exists — the optimal setup varies across tasks *and across individual samples*.

The per-sample variance is the genuinely novel observation, and it is the sort of finding that usually gets averaged away. It implies that reporting "we used GPT-X as judge at T=0" is an under-specification of an evaluation protocol, not a description of one.

BinJudge

The response is **BinJudge**, which "employs a lightweight routing mechanism to adaptively select the optimal judge configuration for each task and sample." Results: correlation with human experts improves by **4.5%–24.7%**, and API cost drops to **0.06×–0.84×** of the static best configuration.

Getting better correlation *and* lower cost is the signature of a routing result done right — most samples are easy and can go to a cheap configuration, and the expensive judge is spent where it changes the answer.

Transferability

Nothing about the routing idea is binary-specific. Any LLM-as-a-Judge pipeline — code review quality, summarisation, test adequacy — faces the same two facts: the best judge configuration is sample-dependent, and judge calls are the dominant cost of large evaluations. A 63.20% ceiling on human correlation is also worth internalising as a sober baseline; it is a substantial improvement over 35.04%, and it is still a long way from a metric one would want to make deployment decisions on unassisted.

2.5 · AGENT RELIABILITY · AgentChaos injects faults at the shared HTTP layer, adapting the classical crash/omission/value taxonomy to LLM API responses👍😐🩹

AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection Authors: Gou Tan, Zhijie Sun, Jieke Shi, Ting Zhang, Zhou He, Qiang Wu, Shuo Liang, et al. **Venue:** arXiv preprint arXiv:2608.06790, 2026 https://arxiv.org/abs/2608.06790

The observation

Agent systems call an LLM API for every response, and those APIs fail in ways that are not the failures the agent's error handling was written for: server errors, truncated responses, corrupted content. Those failures then propagate through downstream agents. The paper's complaint about existing fault-injection methods is that they "are offline, require source code modification, or cannot modify specific response fields" — three separate disqualifications, each fatal for evaluating a system you did not write.

The design move

The insight is a one-liner and it is a good one: **all agent systems access LLMs through the same HTTP interface**, so faults can be injected at that shared layer without modifying source code. Runtime, non-intrusive, and framework-agnostic by construction.

This is the same architectural instinct as papers 1.1 and 1.2 in today's window — find the seam where the interesting behaviour is already observable, and instrument the seam rather than the system. Here the seam is the HTTP boundary, and instrumenting it means one harness works against LangChain, AutoGen, a bespoke orchestrator, or a closed-source product equally.

The taxonomy

The framework "defines a fault taxonomy by adapting the classical fault classification from distributed systems to LLM API responses, covering crash, omission, and value faults on both content and tool call fields."

That mapping is the part worth borrowing even if the tool is not. Crash / omission / value is a forty-year-old classification from distributed systems, and porting it to LLM responses immediately generates the right test matrix: a crash fault is a 5xx, an omission is a truncated or missing response, and a value fault is content that arrives intact but wrong. Splitting the value case across *content* and *tool call* fields is the LLM-specific refinement, and it is the one that matters most in practice — a corrupted tool-call argument is silently executed, whereas corrupted prose is at least likely to be noticed.

Assessment

Positioned here rather than higher because the abstract does not surface the empirical result — how many systems were tested, what fraction of injected faults produced task failure, whether any system degraded gracefully. The framework design is sound and the taxonomy is immediately usable; the evidence of what it *found* is what a full read needs to establish.

This digest's assessment is based on the abstract and metadata; a full-text read is queued.

3.1 · CODE QUALITY · SmellCC is an LLM-based smell-remediation tool reporting cross-project generalisation

SmellCC: A Tool for Automated Code Smells Remediation Authors: Xiaoting Zhang, Yujie Zhang, Zhipeng Gao, Xing Hu, Xin Xia **Venue:** arXiv preprint arXiv:2608.09477 (cs.SE), 2026 https://arxiv.org/abs/2608.09477

Code smells accumulate technical debt, and the practical obstacle is not detection — SonarQube and its peers detect precisely — but that developers lack the time to act on the findings under release pressure. SmellCC ("Smell Code Cleaner") is an LLM-based tool that automatically refactors and removes detected smells, reported as demonstrating strong generalisation in a cross-project setting.

Listed as Borderline High-Priority rather than Keep: it arrives through a followed-researcher alert and the cross-project generalisation claim is the interesting part, but this is a tool paper and the abstract available at run time does not carry the evaluation numbers — remediation acceptance rate, behaviour preservation, or how "removed" is verified — that would let the claim be assessed. Behaviour preservation is the load-bearing question for any automated refactoring tool, and it is the first thing a full read should check.

The framing is worth noting independently of the tool: pairing a precise classical detector with a generative remediator, rather than asking a model to do both, is the same detector/generator division of labour that makes papers 1.1 and 1.2 work.

Assessment based on abstract and metadata; full-text read queued.

3.2 · FAIRNESS · Studies whether reasoning traces amplify or suppress social bias in generated code, beyond direct generation👍😐🦑

How Reasoning Shapes Social Bias in LLM-Generated Code? Authors: Weifeng Sun, Jieke Shi, Zhou Yang, Yuchen Chen, Hongyu Li, Meng Yan, David Lo **Venue:** arXiv preprint arXiv:2608.06829, 2026 https://arxiv.org/abs/2608.06829

LLM-generated programs can encode social bias through unfair or differential treatment of sensitive demographic attributes. Prior work in this area has mainly studied direct code generation; this paper's stated angle is the role of *reasoning* — what happens to bias when the model produces an intermediate chain of thought before emitting code.

The question is well-motivated and non-obvious in both directions. Explicit reasoning could surface and correct an unfair assumption, or it could construct a plausible-sounding justification for one and thereby entrench it. Adjacent work outside the code domain has found the latter effect for some prompt-level interventions, which makes a code-specific study worth having: code is where a biased decision rule gets frozen into an artefact that executes at scale and long outlives the conversation that produced it.

Listed as Borderline High-Priority: the topic sits at the edge of this digest's SE/PL-security focus, and the arXiv record was not resolvable at run time, so only the alert's abstract fragment is available. Findings and evaluation design are unestablished and no numbers are reported here because none were seen.

Assessment based on the alert abstract fragment only; full-text read queued.

Cross-Paper Synthesis

Fifteen candidates and nine survivors is enough to see structure rather than coincidence, and this window has a clear one.

The seam pattern. Three of today's strongest papers share an architecture that is worth naming explicitly, because it cuts against the field's default. Agolic does not put an LLM inside the symbolic execution engine; it puts one outside, deciding how the engine is invoked. PDFuzzer does not ask an LLM to generate test cases; it asks one to read the manual and produce a grammar, then hands generation to a constraint solver. AgentChaos does not instrument agent frameworks; it instruments the HTTP boundary they all share. In each case the design finds a *seam* — a place where the interesting decisions are already externalised — and puts the model there, leaving the deterministic machinery on the other side untouched.

The engineering logic is the same each time. A language model's failure mode is plausible-but-wrong output. Where that output feeds a verifier, a solver, or a bounded run whose results are checkable, the failure is bounded: a bad grammar production wastes search, a bad targeting decision wastes one bounded run. Where the output *is* the answer, the failure is silent and unbounded. The seam pattern is what it looks like when a system is designed around that asymmetry rather than in spite of it. Memoir's

verification gate before memory write-back is the same instinct applied to a feedback loop, and SmellCC's detector/remediator split is a fourth instance.

The evaluation reckoning. The second thread is that this window contains both the most flattering and the least flattering numbers the digest has seen in weeks, and the difference is entirely down to how the evaluation was built. Memoir reports near-ceiling F1 on a curated benchmark. SWE-RPG reports a 31.5% resolved rate for production coding agents, and — because it instrumented the intermediate stages — can say *where* the other 68.5% went. BinJudgeBench reports that the best available automated judge correlates with human experts at 63.20%, a number that reads as a disappointment until compared against the 35.04% that classical metrics manage.

These are not in tension; they are the same observation from three angles. Benchmarks that ask a summary question return summary answers, and summary answers saturate. Benchmarks built to be *diagnostic* — intermediate ground truth, expert multi-dimensional annotation, per-sample analysis — return lower headline numbers and considerably more information. The SWE-RPG finding that implicit-requirement recovery accounts for 24.5–46.0% of agent failures could not have been produced by any pass/fail benchmark at any scale, and it is more useful than another five points of resolved rate would have been.

What this implies for the next few months. If the seam pattern is right, the interesting open problems are not "can a model do X" but "where is X's seam." Bounded model checking, targeted taint analysis, and directed greybox fuzzing all have the properties Agolic exploits — resource bounds, legible traces of what was reached, configurable re-runs — and none has an agentic planner published against it. On the evaluation side, BinJudge's per-sample routing result suggests that any large LLM-as-a-Judge evaluation currently reporting a single fixed configuration is leaving both accuracy and money on the table.

Writing & Rationale Insights

Two observations from producing this report, one about screening under abundance and one about how to write up a number that is too good.

On screening when the window is full rather than empty. This digest has spent most of August writing about zero-candidate days, and the reflex those days build is the wrong one for a fifteen-candidate day. The habit of a quiet window is *verification* — re-query the channel, confirm the emptiness is real, escalate if it persists. The habit a full window needs is *rejection*, and it turns out to be much harder to exercise, because every candidate has some claim on attention and the marginal cost of writing one more paragraph feels small. It is not small. Six of today's fifteen were dropped at Stage-1 — a corrigendum, a survey, a cellular-network fuzzing chapter, a saturated deep-learning vulnerability-detection paper, a weak-venue abstract, and one alert that was a name collision rather than a followed researcher's work — and the report is better for their absence than it would have been with nine more lines of dutiful summary. The rubric earns its keep on days like this one and costs nothing on the empty ones, which is an argument for writing rubrics during the quiet periods.

The preference posteriors deserve a note here too. The feedback base currently has exactly one attribute with more than a single observation, and it is `venue::preprint` at 0.75. Applied literally, that rule auto-promotes essentially every arXiv paper in the window — which is to say it would have disabled Stage-1 screening entirely on the strength of two historical thumbs-ups. It was applied as a tiebreaker on genuinely borderline candidates rather than as an override, and the deviation is recorded rather than quietly taken. A preference system that can be triggered into a no-op by two data points needs a

minimum-observation floor higher than two, and that is a finding about the pipeline, not about the papers.

On reporting a number that is too good. Memoir reports perfect precision and 99.43% F1. There are two easy ways to write that sentence and both are bad. The first is to report it flat, as though a ceiling result on a curated benchmark carried the same evidentiary weight as a field result — which would put it, wrongly, alongside thirty-one confirmed zero-days in shipping software. The second is to editorialise it away, treating the number as prima facie suspect and thereby substituting the reviewer's prior for the paper's evidence.

The approach taken was to report the figure exactly as stated, then say what a benchmark ceiling *means* — that it is a claim about the benchmark's remaining headroom as much as about the method — and then point at the part of the same paper that carries more weight, which is the industrial cross-tool study. This has the advantage of being both accurate and actionable: the reader learns the number, learns why it should not be over-read, and learns which section to open first. It also keeps the criticism attached to an evaluation methodology that the whole subfield shares rather than to any author, which is the correct target. The general rule this suggests: when a result saturates, the useful move is not to doubt it or to launder it, but to find the weaker-looking evidence in the same paper that is actually doing more work, and send the reader there.